Hardware Description Language

Ranim Tom

February 18, 2026

Contents

1 Introduction **11**

1.1 Why HDL . . . 11
1.1.1 Desing methodolgies . . . 12
1.1.2 VHDL . . . 12
1.1.3 FPGA . . . 14
1.1.4 FPGA Configuration . . . 14
1.2 Desing Flow . . . 15
1.2.1 Behavioral Simulation . . . 15
1.2.2 Gate Level Simulation . . . 16
1.2.3 Wrapping up the desing flow . . . 16
1.3 VHDL is not a software . . . 17
1.4 VHDL Desing . . . 18
1.4.1 Entity . . . 18
1.4.2 Architecture . . . 19
1.4.3 Internal Signals . . . 20
1.4.4 Concurrency . . . 20
1.4.5 Process . . . 21
1.5 Some VHDL Examples . . . 22
1.6 Process Rules . . . 24
1.7 More about VHDL Langugage . . . 25
1.7.1 Vectors . . . 25
1.7.2 Bad code: vector mismatch . . . 26
1.7.3 Code comments . . . 26
1.7.4 VHDL syntax exercice . . . 27
1.8 Summary Introduction . . . 28

List of Figures

1.1 Different method to describe a MUX circuit . . . 11
1.2 Desing evolution . . . 12
1.3 VHLD different sets of the language . . . 13
1.4 FPGA device . . . 14
1.5 FPGA device . . . 14
1.6 FPGA development process . . . 15
1.7 Behavioural simulation by using waveforms and set of stimulus . . . 15
1.8 Gate level simulation: output produce by actual gates . . . 16
1.9 Desing flow . . . 16
1.10 VHDL concept . . . 17
1.11 VHDL component . . . 18
1.12 VHDL entity . . . 18
1.13 VHDL architecture with its corresponding entity . . . 19
1.14 Internal signal example . . . 20
1.15 And gate implementation using sequential and concurent style . . . 20
1.16 VHDL process . . . 21
1.17 VHDL NOR gate . . . 22
1.18 VHDL NOR gate . . . 23
1.19 Vectors variables in VHDL . . . 25
1.20 Vectors bad code . . . 26
1.21 VHDL comment . . . 26
1.22 VHDL syntax exercice . . . 27

List of Tables

Todo list

Sensisitivity list 21

Chapter 1

Introduction

1.1 Why HDL

Before starting learning `VHDL`, let's see the idea behind `HDL` in general. Consider a mux circuit shown in Figure 1.1.

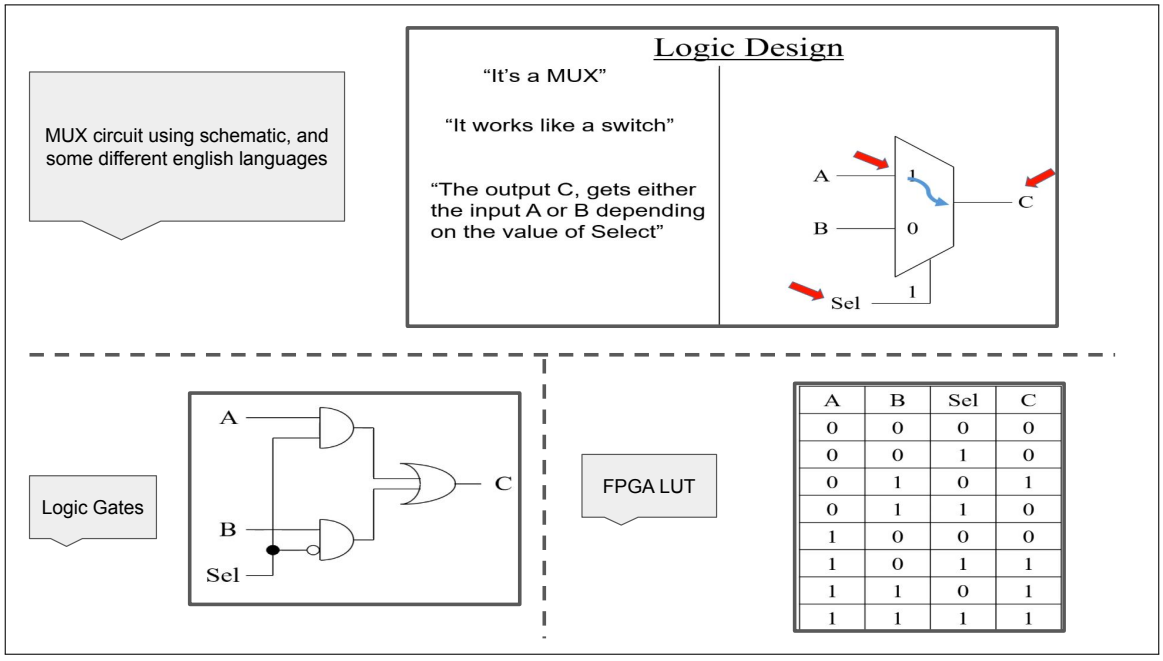


Figure 1.1: Different method to describe a MUX circuit

A MUX can be described using english languages, or the logic gates behind it, or even by LUT. But all these methods are somehow not practical when dealing with some complex design.
The other method which is better to implement our desing is to give some behaviour for the desing (function), and let the tools figure out what logic needs to be used. As long as the tools implement correctly the behaviour, we don't care about the underlying logic. This is the basic idea behind `HDL` such as `VHDL` and `Verilog`.

1.1.1 Desing methodolgies

Throughout time, desing evolves through 3 phases:

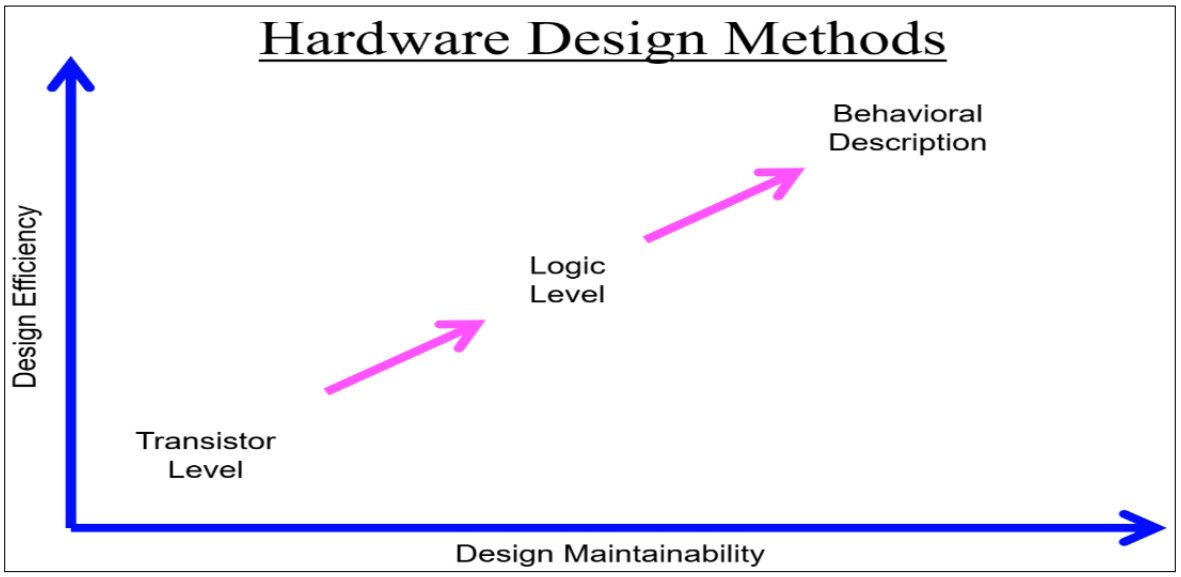


Figure 1.2: Desing evolution

- Connecting transistors manually
- Then use symbol and schematic to simplify the manual connections
  - Such in digital desing courses
- Then move to `HDL` and let the tools figure out the gates and the schematic

1.1.2 VHDL

`VHDL` was created in United states as some defense project for electronic simulation, then later used in logic modeling and simulation.
The most common standard used today is 1993, and recent standard are added after revision in 2006 and 2008.

Simulation and Synthesis:

`VHDL` is a language used for simulation and modeling, and there is allot of features that can't be used in synthesis.

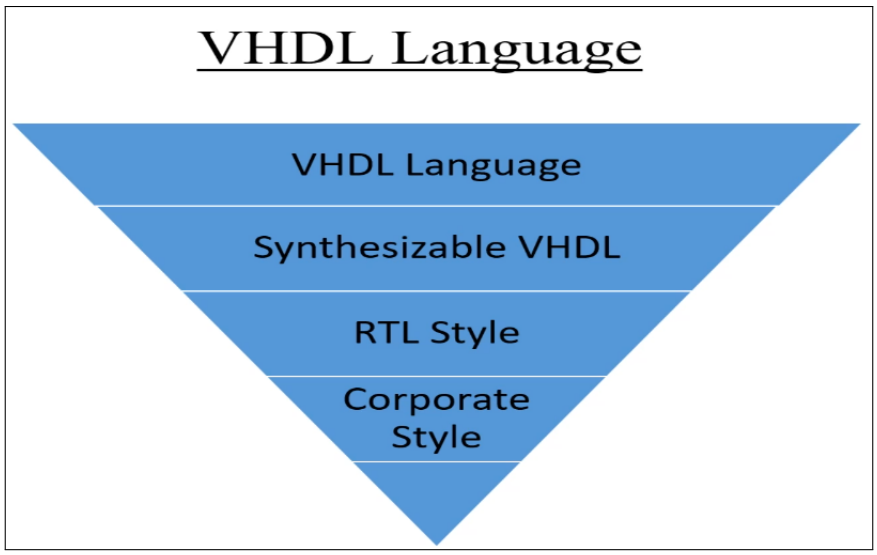


Figure 1.3: VHLD different sets of the language

There is a subset of the language used for synthesis, and we have a specific style used for the logic synthesis called *RTL style*, which will be the focus of this course.
In companies, it is also common to have to whay is called *corporate style*, that is some coding style specific to that company which uses features in certain way. In this course, we focus on Boing style.

1.1.3 FPGA

FPGA is a device composed of many cells, where each cells contains many LUT and FF. The complexity of an FPGA resides in the millions of ways to connect LUT and FF *wihting a cell* and *between cells also*. We use `HDL` and let the tools figures out these connections.

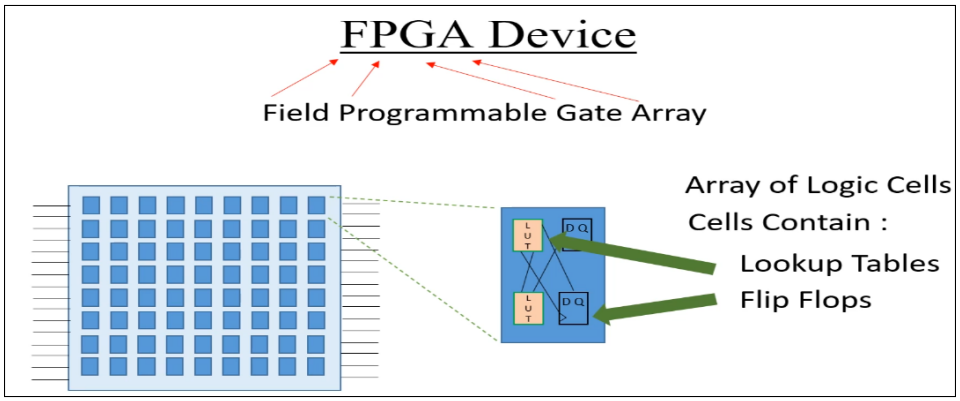


Figure 1.4: FPGA device

1.1.4 FPGA Configuration

By FPGA configuration we mean the way the interconnection is made within the logic cell it self, and from 1 cell to another. This is done by loading these information using some static RAM (or any type of memory), where the SRAM can be transfer using flash memory, or JTAG cable, or some through some processor protocol like `SPI`.
Once these configuration are transfer to make the connections, the connections are made throughout some siwthch boxes, within the cell itself, and from 1 cell to another.

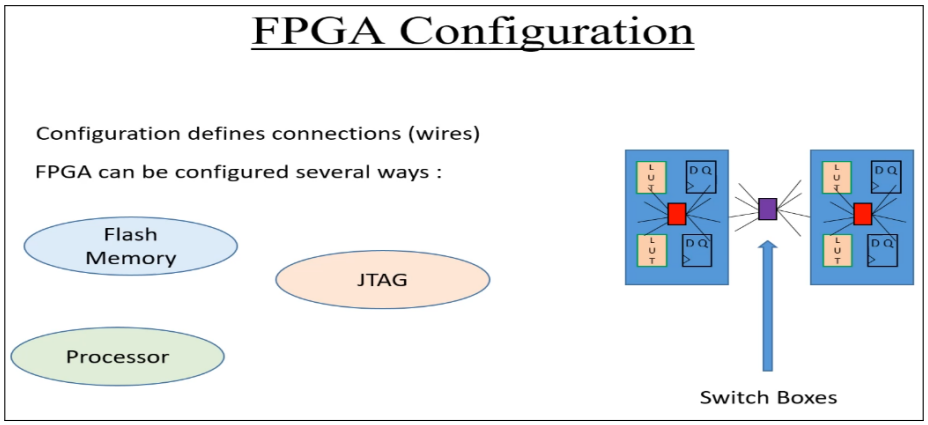


Figure 1.5: FPGA device

This step is defined and implemented is the 3rd step of the FGPA desing flow (1.2), called *place and route*.

1.2 Desing Flow

Now we turn to the FPGA development process. We have 3 phases, as shown in Figure 1.6.

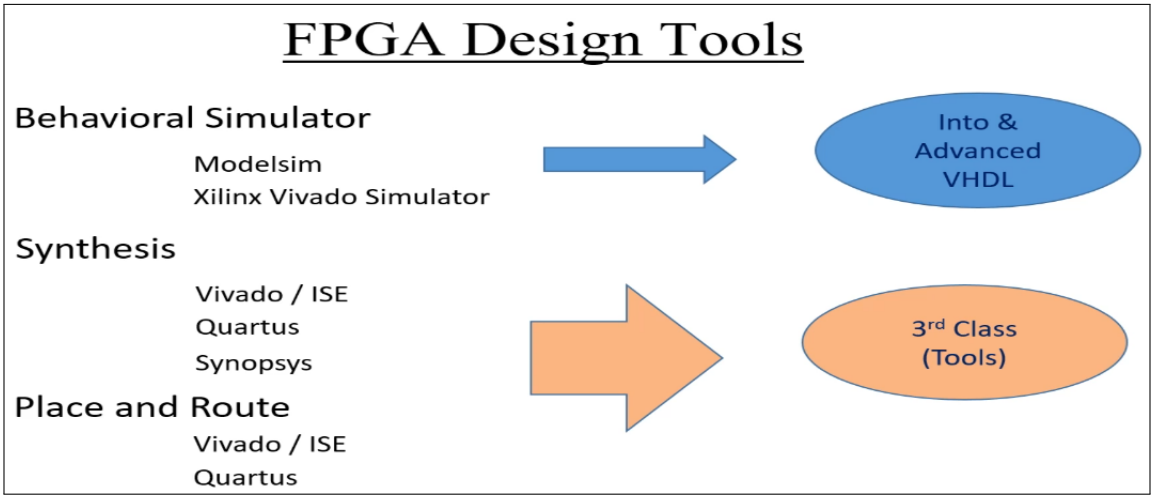


Figure 1.6: FPGA development process

- The 1st step is the *behavioral simulator*, that is we simulate behavioral description of our desing using tools such as `Modelsim` or `Vivado` from `AMD`.
- Once we make sure that the desing simulate the behavior correctly, we translate the behavioral description into logic.

  This is done using *synthesis tools*, such as `Vivado` also.
- 3rd step: place and route

1.2.1 Behavioral Simulation

In the 1st part of the course, we will focus on behaviour simulation.

The idea behind behavioral simulation is that we are simulating the behaviour of the `VHDL` code, and not the logic synthesised yet.
This is done using waveform visualisation, that is giving our code a set of input (stimulus), and then we hope that our code produce the correct result, and be able to visulaize this using the waveforms.

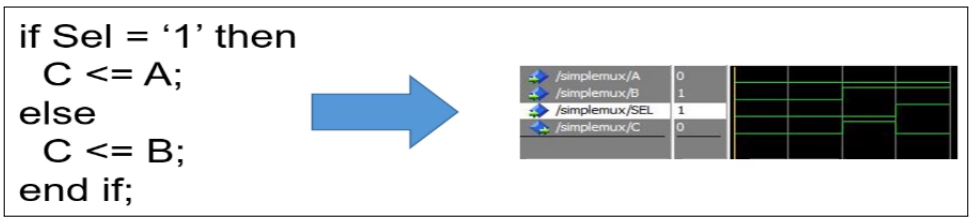


Figure 1.7: Behavioural simulation by using waveforms and set of stimulus

1.2.2 Gate Level Simulation

This is the part we simulate tha actual logic synthesis by the tools. The goal is to hope that gate simulation mathces the behaviour simulations. It is always better to do both, and with experience we gain some confidence that our `HDL` code produces the correct gate simulaitons.

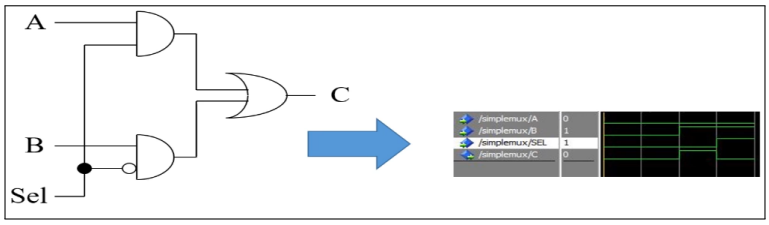


Figure 1.8: Gate level simulation: output produce by actual gates

What can go wrong?

- Sometimes we describe things that can't be translated into logic
- After synthesis, timing problems.
  - We learn how to correct these issues in another part of the course

1.2.3 Wrapping up the desing flow

Figure 1.9 presents another view of the development process seen in Figure 1.6.

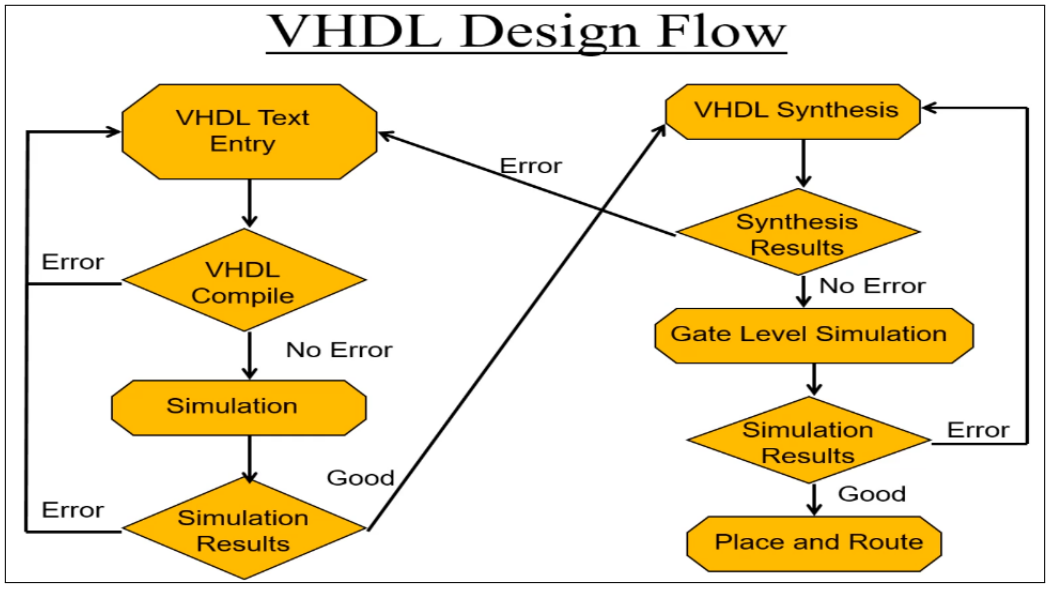


Figure 1.9: Desing flow

- Before the running the simulation part (behavioural simulation as see in 1.2.1), there is the compilation step where we can check for errors like syntax errors, langauges rules,$\cdots$
- Same concept for gate level simulation, we have to pass also the synthesis result step

1.3 VHDL is not a software

It is important to keep in mind is not a traditional programming language, that is the final output is not binary or some executable like a `C` program, which runs on some machine. As a matter of fact, we need to think of `VHDL` as some kind of desing tools which let us *describe the behaviour of our desing*, and then using vendors tools we synthesis this behavour into actual gates and circuit.

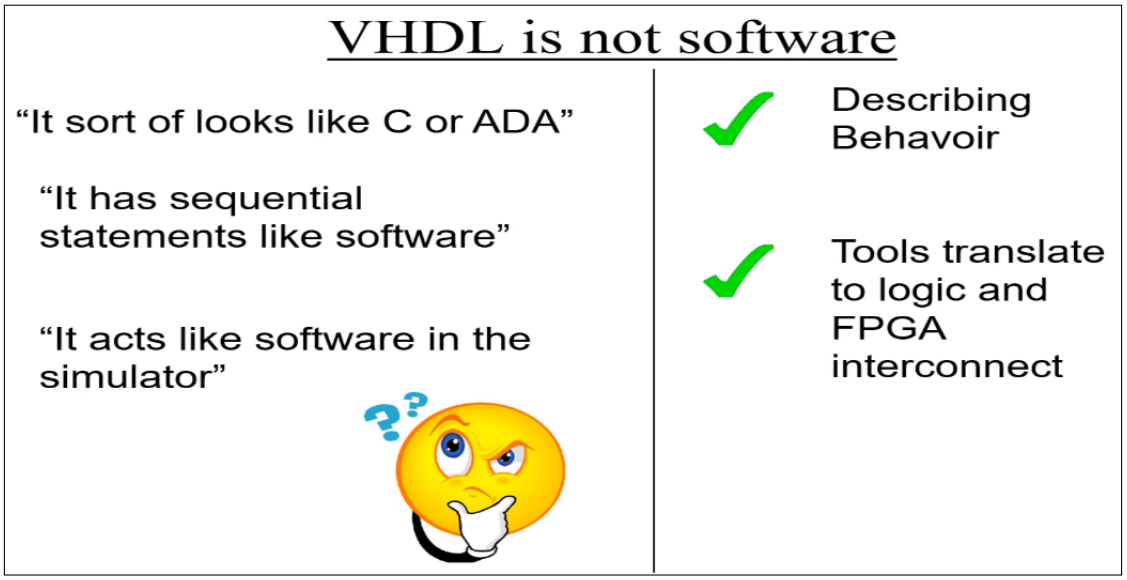


Figure 1.10: VHDL concept

So in other words, `VHDL` is a tools which produces circuit, and not some binary.
After experience, we can imagine what is the circuit behind our `VHDL` code lines .

1.4 VHDL Desing

Now we move more toward the desing itself and VHDL language.

There are 2 blocks to have a correct VHDL module as seen in Figure 1.11:

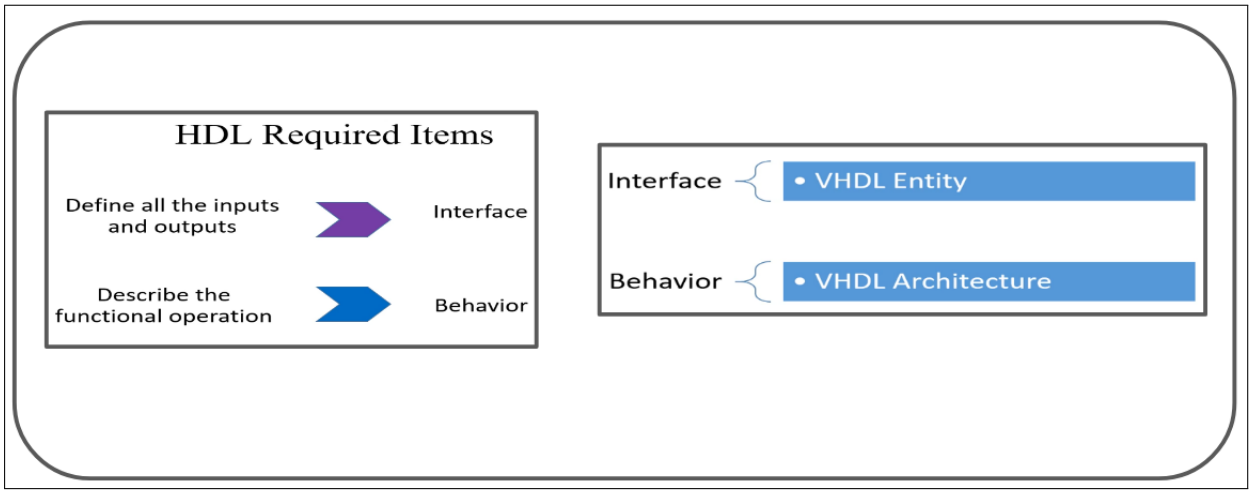


Figure 1.11: VHDL component

- The input and output part, which are structured in *entity* unit
- The behavioral part, that is the actual work of the system, and it is contained in *architecture* unit

In VHDL vocab, a complete VHDL desing which is formed by an entity and a architecture is called *component*.

1.4.1 Entity

The entity contains all input output of our desing. Figure 1.12 shows the parameter and syntax of VHDL entity

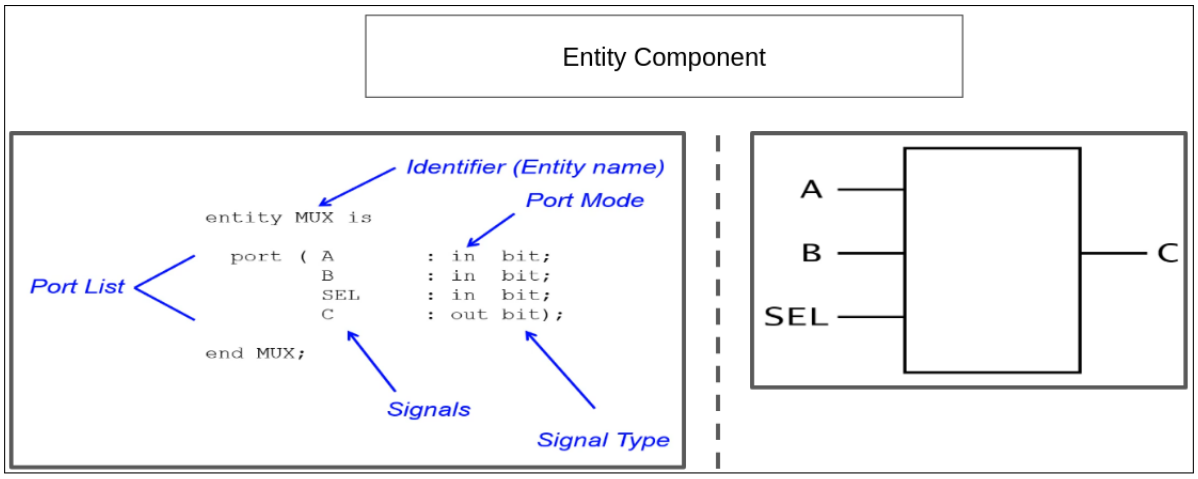


Figure 1.12: VHDL entity

- We begin by the keyword `entity` and its name (`MUX`)
- `port()` contains a list of all input and output
- we type their name (`A`) followed by `:`
- Their mode `in` or `out`
- Their type `bit`
- Finally we use the `end` and the name `MUX`

Some notes:

- In `VHDL`, an entity must be associated with some architecture, where we code our behaviour.
- From a syntax: note that in Figure 1.12 that the last `;` is placed after the parenthesis, and not before

1.4.2 Architecture

The architecture is where we define our functional desing.

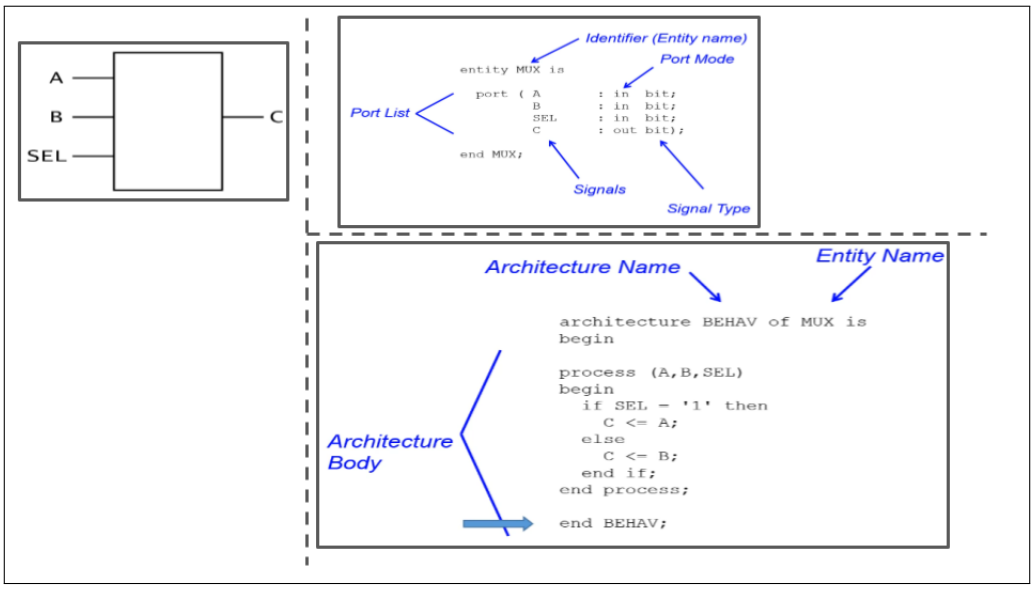


Figure 1.13: VHDL architecture with its corresponding entity

- The 1st line is the *architecture declaration*, where we have the archi name (`BEHAV`) and the associated entity (`MUX`)
- After declaration, we use the keyword `begin`
  - Inside this `begin` block, we put our code, that is the behavioural description of our system
- In the case of a MUX circuit, we put our behavioral description inside `process` block, where we have some sequential statemets, that is statement executed one after another

1.4.3 Internal Signals

Any signal (like a variable) wich is defined in the entity can be used inside the archtiecture. However, sometimes we can use some internal signals, that is signal which are only visibile inside the architecture unit. An example is shown in Figure 1.14.

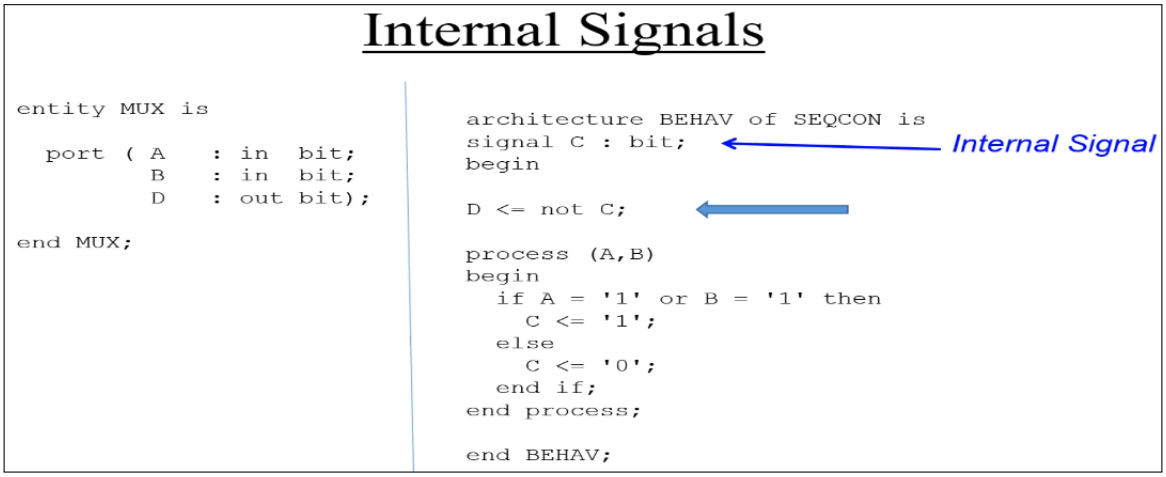


Figure 1.14: Internal signal example

- signal `C` is declared inside the architecture, but in the entity
- It is assigned to `D` by wiring up to a `NOT` gate
- Then it is executed both in parallel with the process

1.4.4 Concurrency

The statement defined outside a process but inside the architecture are executed in parallel with the processes. Figure 1.15 present an example of `AND` implementation

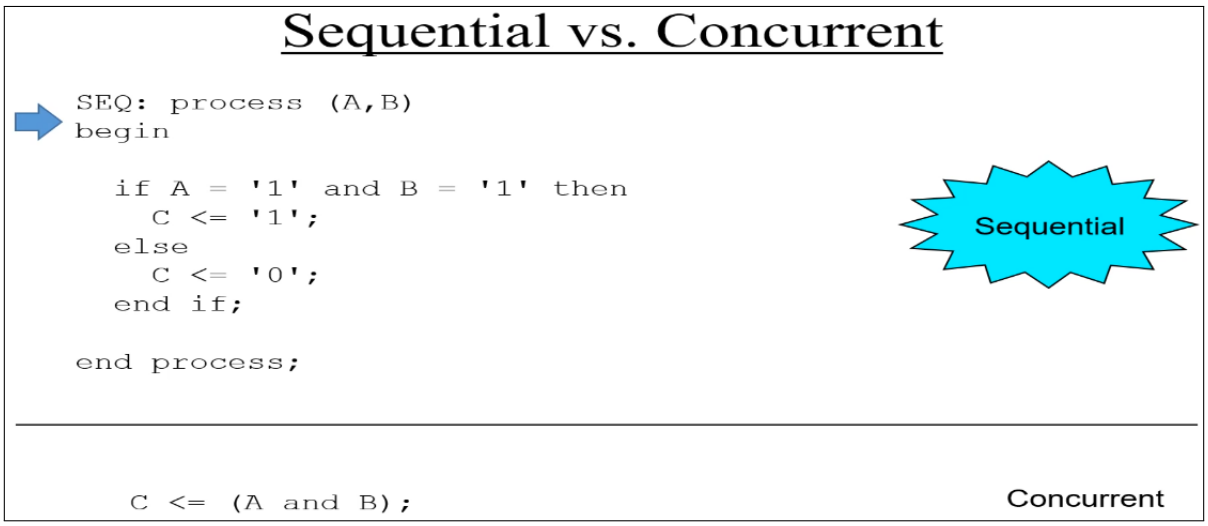


Figure 1.15: And gate implementation using sequential and concurent style

- 1st implementation sequential style using `if-else`
- 2nd style using `AND` keyword statement , which is concurrent

As a desingers, we use a combination of both sequential and concurrent statements. Also note that the order of execution of concurrent statement is not important, since they will all be executed in parallel using the capacity of FPGA.

1.4.5 Process

As we saw processes contains statements that are executed sequentially. Figure 1.16 presents paramter of `VHDL` process.

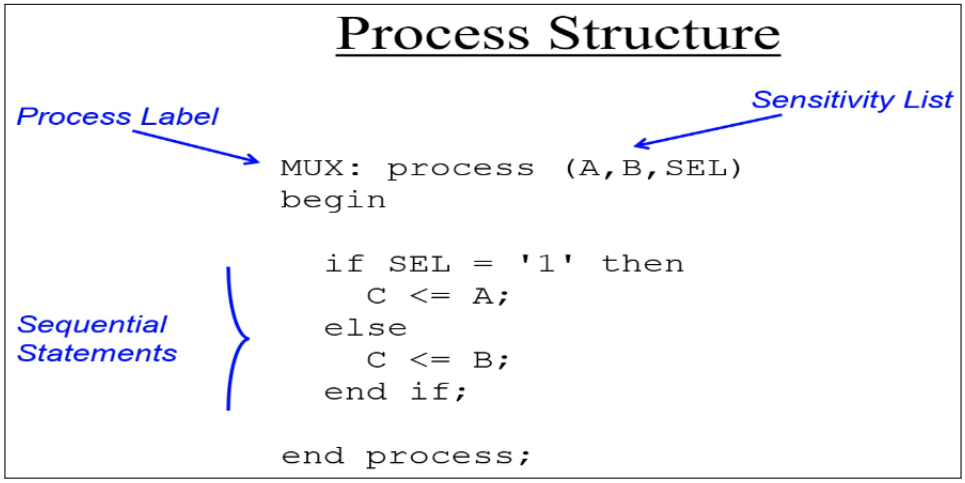


Figure 1.16: VHDL process

- Sensitivyt list is used when a process will be executed
  - It is executed when at least 1 of the input is changed
- Later we will see why we use sensitivity list in `VHDL`
- *Sensisitivity list: To get back to it later, and cite the corresponding section in the reader*

Sensisitivity list

1.5 Some VHDL Examples

In this section we take some example of some VHDL code to get more practice.

- Figure 1.17 present the code of a NOR gate

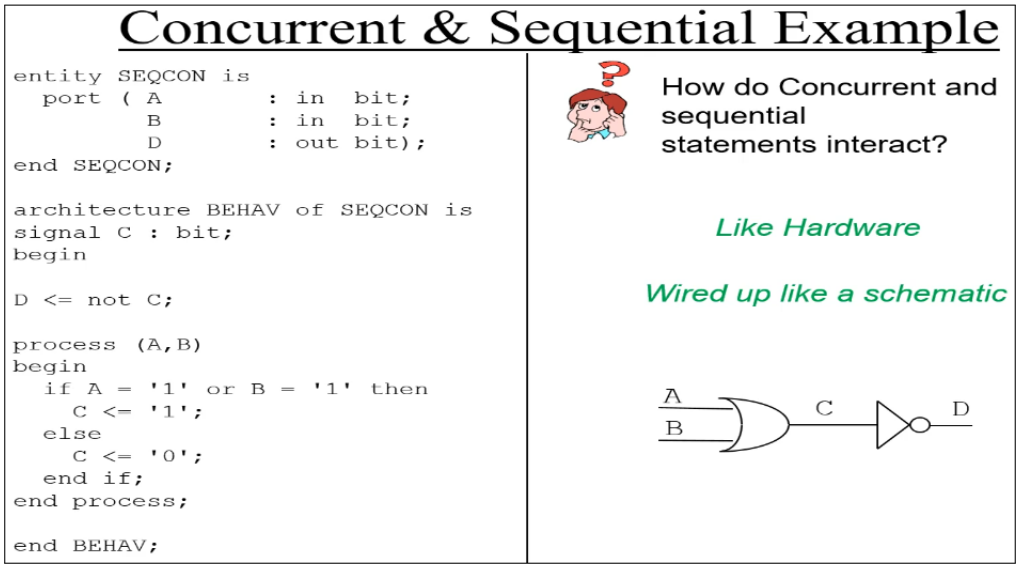


Figure 1.17: VHDL NOR gate

  - Like we use wire to connect circuit in schematic, in VHDL we use `signal` to do the connection
  - The code of Figure 1.17 present an example of sequential statement interacting with concurrent statement `D <= not C`

- Figure 1.18 presents an example of using 2 same statements, but one outside a process (inside the architecture), and the other inside a process

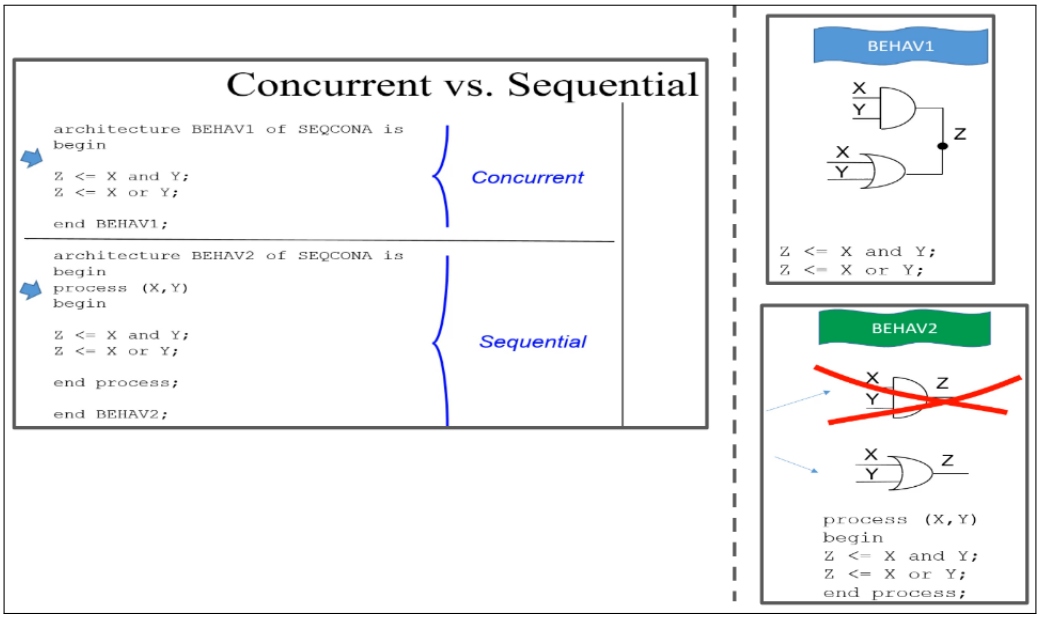


Figure 1.18: VHDL NOR gate

  - This illustrate the difference between sequential and concurrent code
  - In the 1st behaviour, the same set of input (of logic) are driving the same wire. This is isn't good because it causes a *contention*
  - In the 2nd beahviour, it turns out that the first statement get ignored, and only second statment get executed

Actually this one of the `process` rules, which states that:

  - Only last valid assignement to a signal takes place inside a `process`
  - That assignment happens at *the end* of a `process`

1.6 Process Rules

Let's see the `process` rules in this section.

- Process executes when inputs in the sensitivity list change
- Process can only contains sequential statements
- Signal assignment occurs at the *end* `process` statment
  - They don't happen when the statement get executed, they only happen at the end `process`.
- Signal get the last executed assignment

These rules are a little bit of strang at first, but we will revisit them always and need to remember them, since these rules affect our way of designing our code and the sequential statement inside the `process`.

1.7 More about VHDL Langugage

Throughout our progress, we learn more `VHDL` concept and features as needed, so we can grasp concept in a sequential manner.

1.7.1 Vectors

First let's start with signal that are larger than 1 bit, which are `bit_vector`. `bit_vector` are group of bits, or from a programming side array of bits.

An example is shown in Figure 1.19.

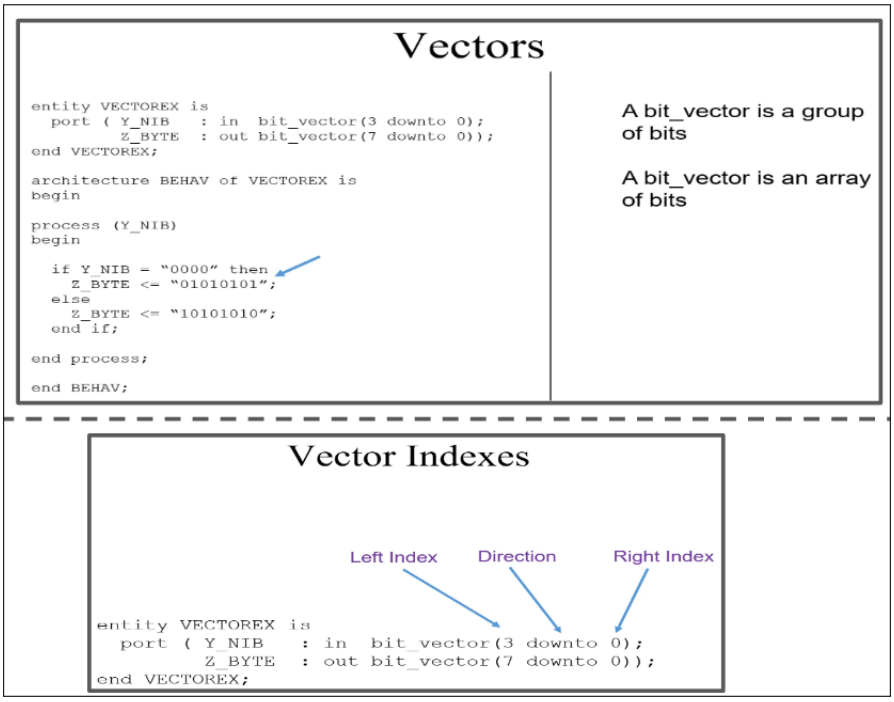


Figure 1.19: Vectors variables in VHDL

- We have `Y_NIB` and `Z_BYTE`, 4 and 8 bit vectors respectevely
- Note for vectors we use double quotation `""`, as oppsed to signal which use single quotation `''` for 1 bit case
- For indices: they are going from left to right as shonw in the 2nd part of Figure 1.19, with a description in the middle `downto` or `upto`

1.7.2 Bad code: vector mismatch

Nice thing about VHDL it is a strict languafe, it won't let you assign vectors to different size, or of different type. An example of bad code is shown in Figure 1.20.

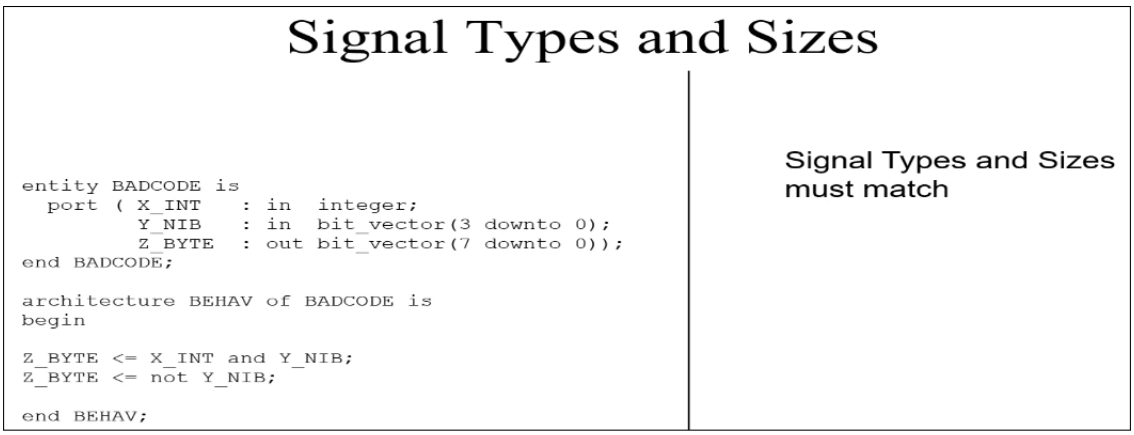


Figure 1.20: Vectors bad code

- We have 2 concurrent statements
- The 1st statement we get a compile time error since we are trying to assign to Z_BYTE from 4 vector, and also a wrong type variable (X_INT is integer)
- The 2nd statement is also wrong since Y_NIB is 4 bit vector, whereas Z_BYTE is 8 bit vector

1.7.3 Code comments

In VHDL comments are insetered after 2 dashes --. An example is shown in Figure 1.21.

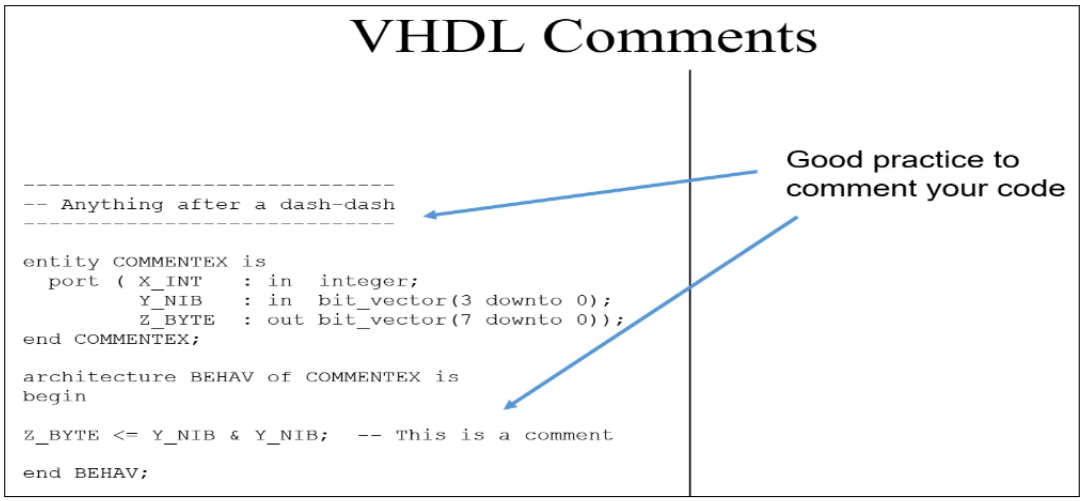


Figure 1.21: VHDL comment

However in VHDL, there isn't a block comment feature as in some other programming language.

1.7.4 VHDL syntax exercice

VHDL is a strict language, and we often need time and practice to get use to it. If we make some mistakes, the compiler gives errors, but it will be hard to find where these error happens, so we need to practice on the syntax and rules.

Given: Figure 1.22 contains a VHDL code with some errors. Try to spot the errors.

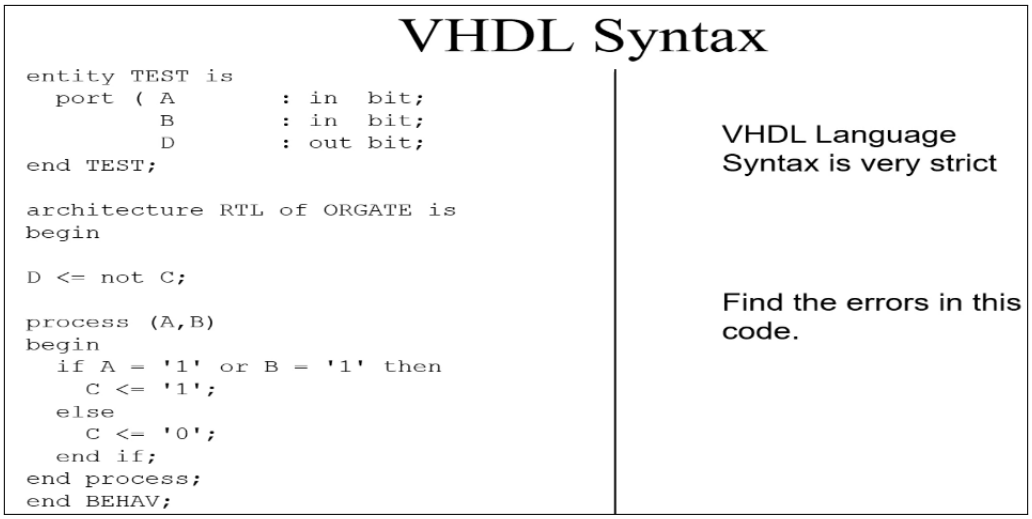


Figure 1.22: VHDL syntax exercice

Solution:

- 1st error in the `entity` declaration, we are missing the last parenthesis before the last semicolone
  - ↔ `D:out bit);`
- 2nd in the `architecture` we are referring to the wrong `entity` name
  - It should be `TEST` instead of `ORGATE`
  - ↔ `architecture RTL of TEST is`
- Signal declaration is missing for variable `C`
  - it should be declared after the `architecture`, and before the 1st `begin` statement
  - ↔ `signal C : bit` (see 1.4.3 and Figure 1.14)
- At last, we are ending with the wrong architecture name
  - it should be `RTL` instead of `BEAHV`
  - ↔ `end RTL`

1.8 Summary Introduction

- Basic idea behind `HDL`: give our desing some desing or behaviour, and let the tools figure out the underlying logic and gates (1.1)
- Desing evolution: transistor $\rightarrow$ gates and logic $\rightarrow$ `HDL` (1.1.1)
- The output of a `VHDL` (or any `HDL`) is a circuit, and not a binary or some executable like known programming languages (1.3)
- Development process of an FPGA (1.2): we have mainly 3 steps

  1. Behavioral simulation: writing code, compiling the code then verify the behaviour using a set of stimilus and visualise them by waveforms
  2. Gate level simulation: which include synthesis verification, then checking if output of logic gates is correct or not
  3. Place and route: make the connections within and between FPGA cells (review 1.1.4 for more details about connections)

Now regarding `VHDL` (1.4):

- From a language desing:
  - Mainly a `VHDL` program consist of 2 parts: entity and architecture
  - Entity: we list input and output, and all their relative information (their types, modes,$\cdots$)
  - Architecture: contains the body of our code, and consist of describing how the system work
- For the architecture:
  - Anything after the architecture delaration is executed in parallel
  - An architecture is a combination of many processes, internal variables declaration, $\cdots$
  - A `VHDL` rule: an architecture must associated to some entity
- In a `process`
  - all the code is executed sequentially
  - The process should contains a sensitivity list $\leftrightarrow$ `process(A,B)`
  - We say that this process is *sensitiv* to `A` and `B`
- A `VHDL` code usually consist of a combination of sequential and concurrent statement.
  - Order of execution of concurrent statement is not important, since all statement are executed simulatenously, using FPGA (hardware) capacity (1.4.5)